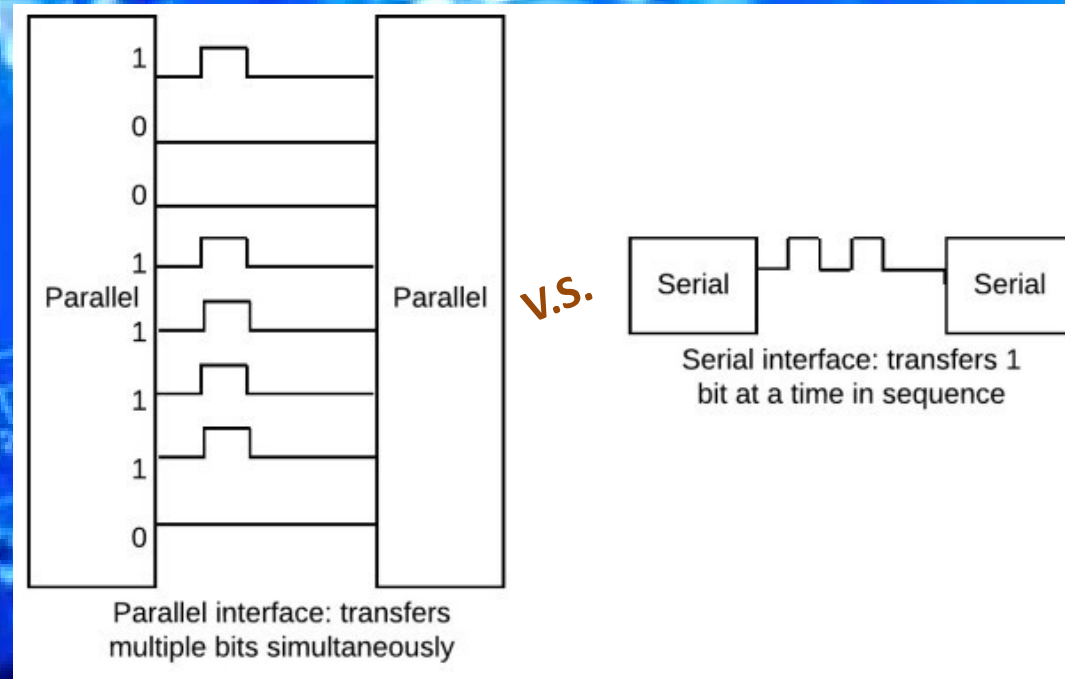


EEE3096S



Embedded Systems II



Embedded Communications 1/3

Embedded Systems II

L09

Mr. Nkhumeleni Best



Electrical Engineering
University of Cape Town

Outline of Lecture

- Intro to embedded communications

Plan for today:

- Quick training (or 'review' for most of you probably) regarding "Embedded Communications essentials"
- Diving into I2C and a few semaphore-answered questions

Acknowledgement: these slides adapted from ESII 2021 slides by J. Wyngaard.

ES Communications

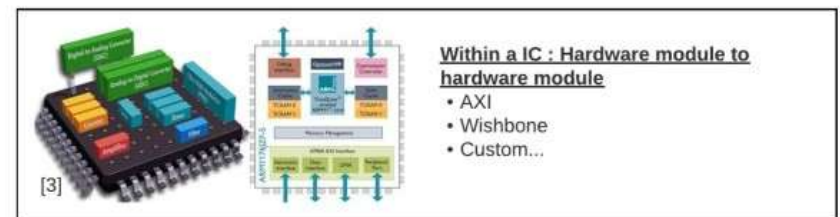
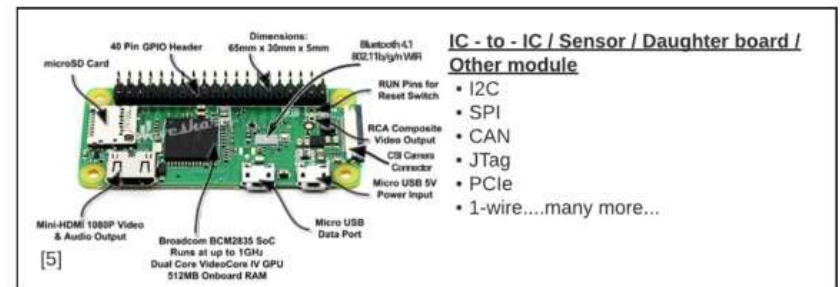
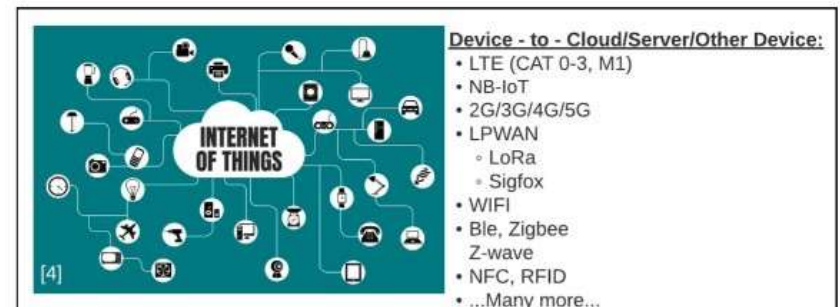
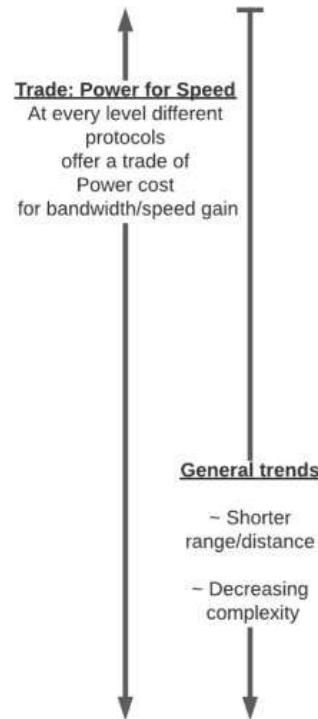
- Comms 1:
 - Intro: Serial vs Parallel, Synch/Asynch
- Comms 2:
 - I2C & SPI
 - PWMs & Peripherals
- Comms 3:
 - Timers
 - RS232 & RS485
 - Briefly, some other interface protocols
 - {Brief Introduction to Wireless Communications – if time permits}

EE Communications

Reminder: Definition of an embedded system:

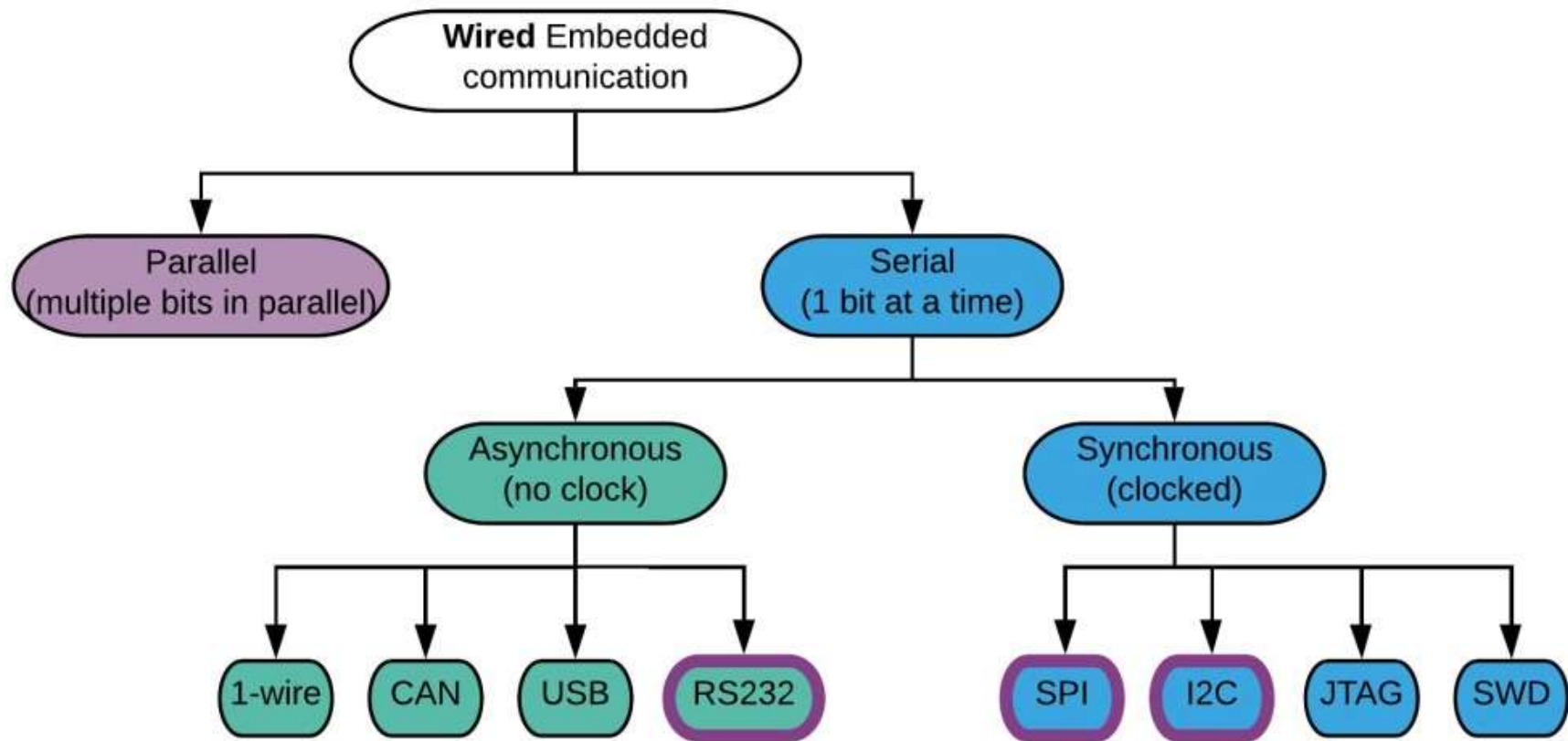
An ES is a task-specific computer, built into a larger system for the purpose of controlling and monitoring the larger system. (i.e., as per earlier “Brisbane” definition [John Catsoulis])

- A microcontroller or microprocessor *is just 1 part* of that larger system
- These micros need to be able to interact with the larger system and the physical world
- This is do via either or both wired and wireless interfaces



View of a wired ES comms interface

Classification hierarchy of wired communications, formal standards at bottom

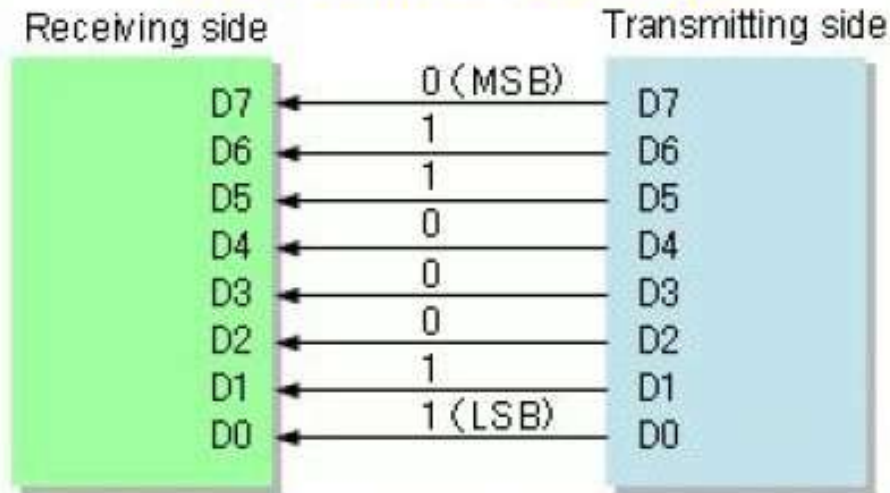


In this course we will look at serial standards of:

UART / RS232 (quite similar but wiring and voltages for RS232 are more specific),
Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) protocols

Parallel vs Serial Communications

Parallel interface example



Serial interface example (MSB first)

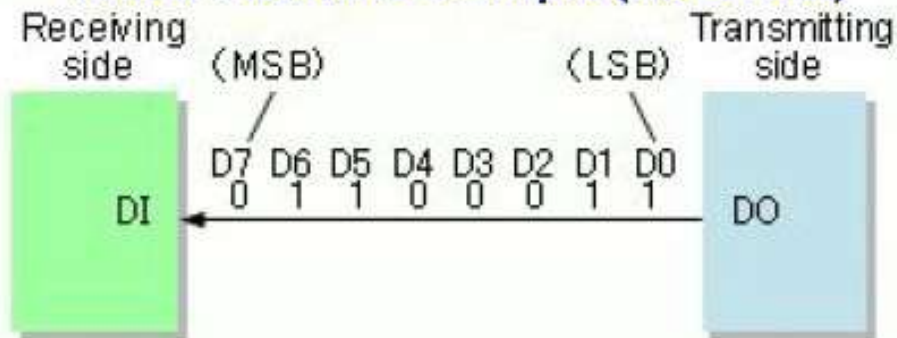
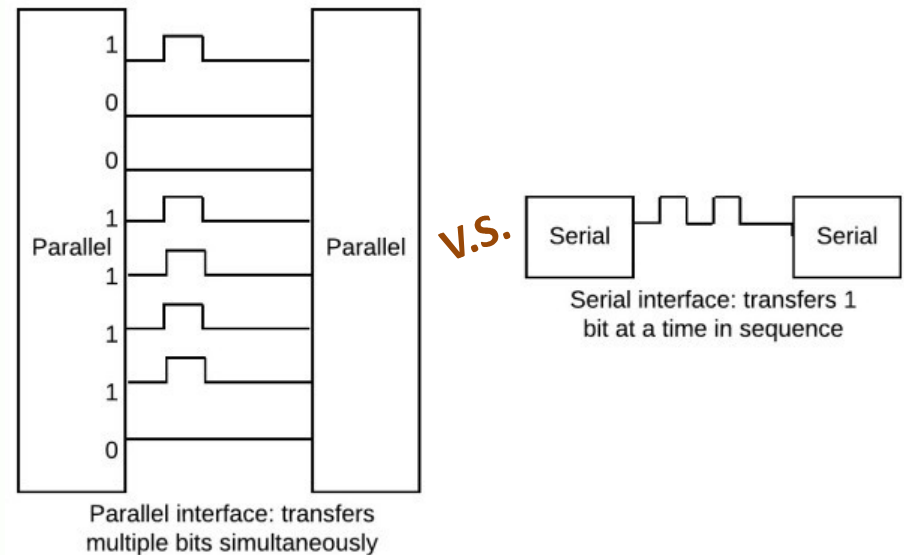


Illustration of data waveforms in time

illustrating that parallel sends multiple bits in one time unit vs. serial sending a bit at a time.



Serial vs Parallel Interface structure

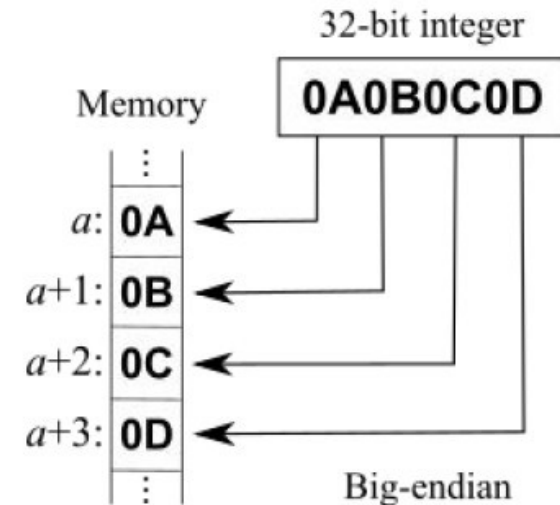
Parallel sends multiple bits of data in one go on separate lines; whereas serial sends one bit of data on one line at a time*.

* actually, that is the simplest half-duplex form, serial may be full-duplex and possibly send multiple bits in parallel but not technically a whole word of data in one moment.

Wired Comms Endianness

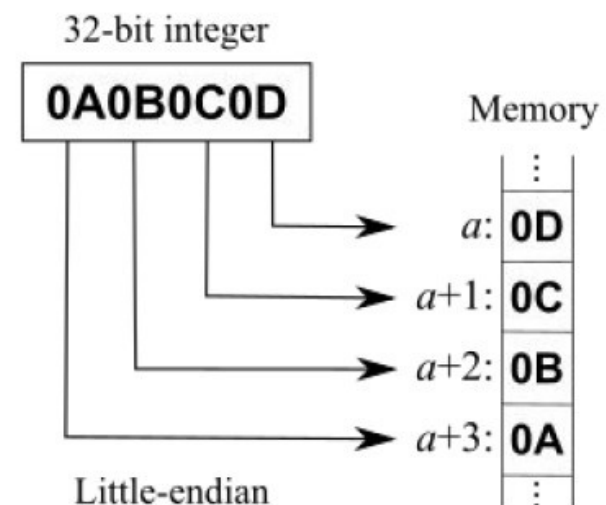
Endianness = the order or sequence of bytes of a word of digital data in computer memory (or in a data register).

- **Big-endian (BE):** The most significant byte (MSB) of a word is stored at the smallest memory address and the least significant byte (LSB) at the largest.
- **Little-endian (LE):** The LSB is stored at the smallest address.



For data transmission:

- **Big-endian (BE):** Means the most significant bit is transmitted first
- **Little-endian (LE):** Means the most significant bit is transmitted last



Endian Examples

Big Endian

Address → Value

0x00	12
0x01	34
0x02	56
0x03	78

(MSB 12 is stored first)

Little Endian

Address → Value

0x00	78
0x01	56
0x02	34
0x03	12

(LSB 78 is stored first)

Endian use cases

Big endian uses

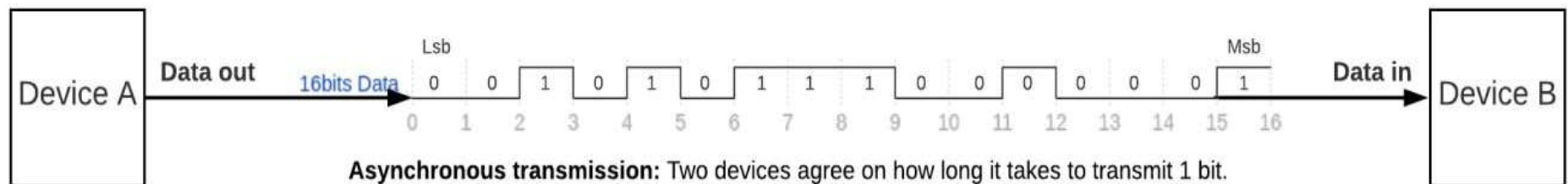
- **Big Endian Systems (less common today)**
- **Old Motorola 68k processors** (used in early Apple Macs, Amigas, etc.)
- **SPARC** (used in some Sun/Oracle servers)
- **IBM PowerPC (older Macs, game consoles like PS3, Xbox 360, Wii)** — typically big endian
- **Mainframes** (IBM z/Architecture) often use big endian
- **Networking protocols** (TCP/IP, IPv4/IPv6 headers, etc.) always define data in **big endian** (that's why we call it *network byte order*).

Little endian uses

- **Little Endian Systems (most common today)**
- **Intel x86 / x86-64 CPUs** (all modern desktops and laptops — Windows, Linux, macOS on Intel chips)
- **AMD processors** (since they're also x86-64)
- **RISC-V** (default is little endian, though spec allows both)
- **ARM (most modern)** — runs in little endian mode by default on phones, tablets, Raspberry Pi, etc.
- 🙌 Basically, **your laptop or phone is almost certainly little endian.**

ES Wired Comms: Asynchronous

Asynchronous = un-clocked in electronics terms
i.e. it does not have a separate clock (or clock somehow incorporated) that is used to indicate when actions or decisions occur, e.g. when to check if the signal is high or low.



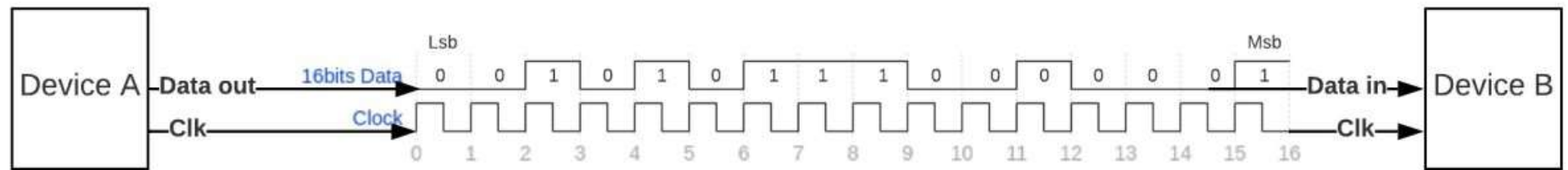
More formal definition of asynchronous communications *:

In telecommunications, asynchronous communication is transmission of data, generally without the use of an external clock signal, where data can be transmitted intermittently rather than in a steady stream. Any timing required to recover data from the communication symbols is encoded within the symbols. Wikipedia

* source: https://en.wikipedia.org/wiki/Asynchronous_communication

ES Wired Comms: Synchronous

Synchronous = clocked in electronics terms
i.e. it means that a separate clock (or clock somehow incorporated) that is used to indicate when actions or decisions occur.



Synchronous transmission: Two devices agree to use a shared clock signal to interpret timing.

More formal definition of asynchronous communications *:

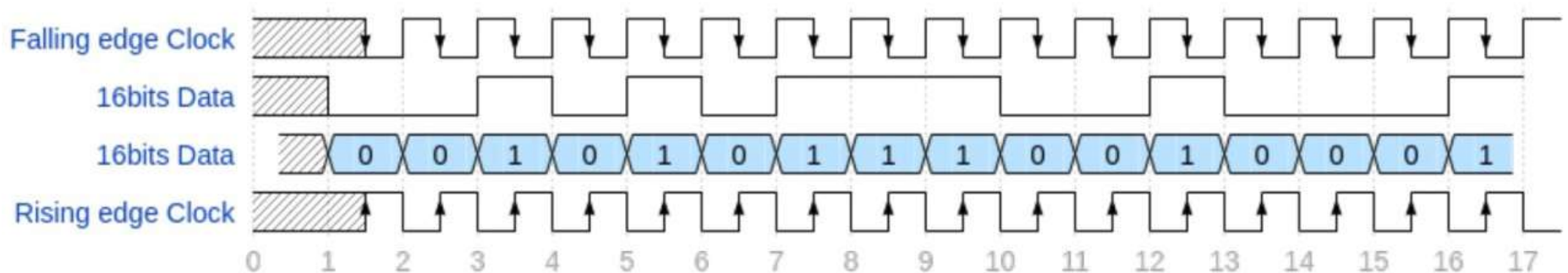
In telecommunications, synchronous communication requires that the clocks in the transmitting and receiving devices are synchronized – running at the same rate – so the receiver can sample the signal at the same time intervals used by the transmitter. No start or stop bits are required.

Wikipedia

* source: https://en.wikipedia.org/wiki/Synchronous_serial_communication

Sampling on an edge?

As mentioned in the definition for synchronous communication, it indicates that a decision or action with regards to the data is actioned on a signal, which is usually a clock edge (it could, for asynchronous, be an agreed time delay).



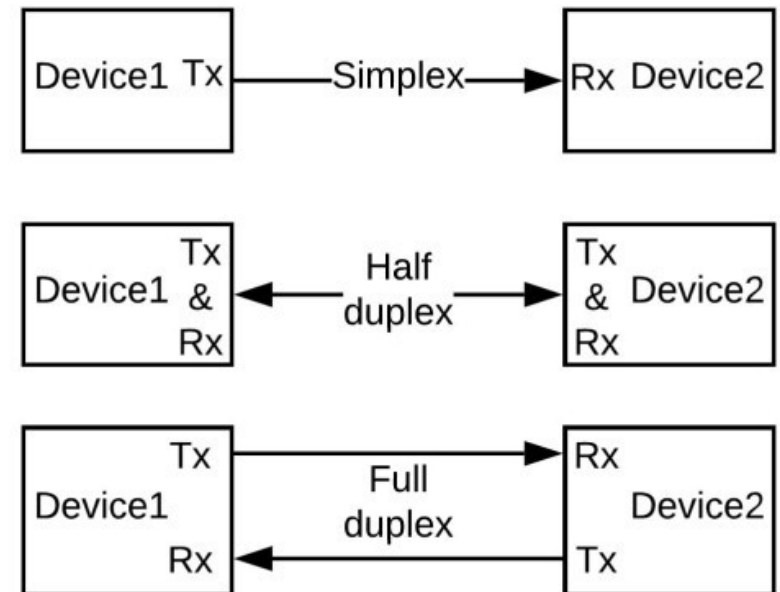
This image illustrates the difference in rising and falling edge. The 2nd waveform down can be considered the data line coming in to the receiver. The clock signal is shown on the 1st and 4th waveform, the 1st indicating when a falling edge occurs, and the 4th when a rising edge occurs. If you sample the input line on a falling edge you may get different results to sampling the line on a rising edge. This, protocols need to specify if, for a synchronous protocol at least, if the protocol is rising or falling edge triggered*.

* hint: this can make for a convenient test or exam question!

Wired Comms: Simplex vs Duplex

Endianess = the order or sequence of bytes of a word of digital data in computer memory (or in a data register).

- **Full Duplex:** Two devices can communicate simultaneously
 - Separate data lines for transmit (Tx) and receive (Rx)
- **Half Duplex:** Two devices communicate, one at a time
 - Transmit and Receive share the same data line
- **Simplex:** Two devices communication in one direction only
 - Unidirectional communication (ie. only transmit or only receive)



Parity bit and checking

A parity bit (or check bit) = a single bit added to a string of binary code to provide a low-cost form of error detection.

- Usually applied to the smallest unit of transmission in a communication protocol (e.g. a single byte)
- **Even parity:** The sum of '1's in desired data including the parity bit must be an even number
- **Odd parity:** The sum of '1's in desired data including the parity bit must be an odd number

7-bit data word	Count of 'ones' in data word	Final 8-bit word transmitted including parity bit	
		Even parity	Odd Parity
1010101_	4	10101010	10101011
1111111_	7	11111111	11111110
0000000_	0	00000000	00000001
1001111_	5	10011111	10011110

ES Wired Comms : Summary

Other Serial interfaces you've heard of:

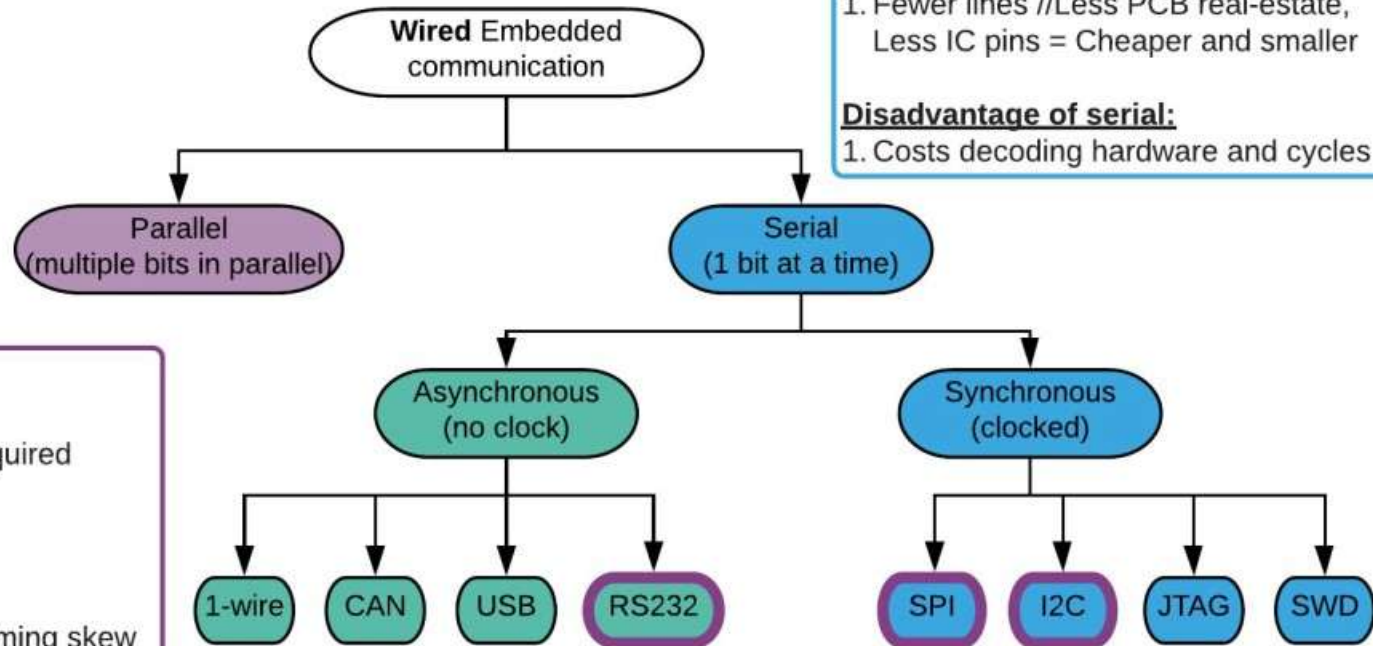
1. Displays: HDMI, DVI
2. Harddrives: SATA
3. Thunderbolt
4. PCI-Express (PCIe)

Advantage of serial:

1. Fewer lines //Less PCB real-estate, Less IC pins = Cheaper and smaller

Disadvantage of serial:

1. Costs decoding hardware and cycles



Advantage of parallel:

1. Easy/Simple to use
2. No decoding hardware required

Disadvantages of parallel

1. Half-duplex
2. Excess signal count
3. Lower bandwidth due to timing skew

References

[1] Endianness

<https://en.wikipedia.org/wiki/Endianness>

[2] Parallel communication

https://en.wikipedia.org/wiki/Parallel_communication

[3] Classification of Embedded Systems with Applications

<https://www.watelectronics.com/classification-of-embedded-systems/>

[4] Importance of IoT in Education

<https://ilmiversity.com/importance-of-iot-in-education/>

[5] Raspberry Pi Zero WH, built-in WiFi, presoldered headers

<https://www.waveshare.com/raspberry-pi-zero-wh.htm>